

전자전기컴퓨터설계실험II

LAB 1. 조합회로 실습

VS Code 시뮬레이션 · Vivado · FPGA 구현



서울시립대학교
UNIVERSITY OF SEOUL

작성일 2026. 09. 10.
이해리

LAB 1 목차

1. 조합회로 10개: 입출력·진리표·Verilog 설계
2. VS Code 시뮬레이션과 실험 전 레포트
3. 레거시 / Vivado 2026.1 회로별 실습
4. 통합 프로젝트: 버튼 모드 선택·LCD 표시
5. FPGA 기록·사진·영상·GitHub와 실험 후 레포트
6. macOS ARM64용 오픈소스 CLI 통합 실습

항목을 누르면 회로 목록·프로젝트 구성·레포트 안내로 이동한다.

조합회로 실습 10개

- | | | | |
|----|-------------------|----|------------|
| 01 | AND·OR·XOR 논리 게이트 | 06 | 4:1 멀티플렉서 |
| 02 | 전가산기 | 07 | 1:8 디멀티플렉서 |
| 03 | 4비트 가산기 | 08 | 8:3 인코더 |
| 04 | 4비트 감산기 | 09 | 3:8 디코더 |
| 05 | 4비트 크기 비교기 | 10 | 7세그먼트 디코더 |

전가산기는 반가산기 두 개를 연결하여 구성한다.

각 회로에서 예상값·시뮬레이션·실제 보드의 동작을 비교한다.

실습 환경과 프로젝트 구성

환경	개수	실습 내용
레거시 Vivado	10	원본 교안에 맞춘 회로별 구현
Vivado 2026.1	10	최신 환경에서 회로별 구현
Vivado 2026.1 통합	1	버튼 모드 전환·LCD 표시
오픈소스 CLI 통합	1	같은 통합 RTL로 비트스트림 생성
합계	22	각 프로젝트에 전용 workspace

- workspace에서 소스·테스트벤치를 열어 시뮬레이션한다.
- 통합본은 비트스트림 하나에서 모드 01-10을 선택한다.
- CLI: macOS ARM64 배포 확인, WSL에서 실행 검증.

실험 전·후 레포트의 범위

실험 전: VS Code

1. 회로 원리·입출력·예상값
2. 설계 코드·테스트벤치
3. 시뮬레이션·로그 확인
4. 파형과 예상값 비교

실행 방법과 파형 해석을
실험 전 레포트에 자세히 쓴다.

실험 후: Vivado·보드

1. Vivado 시뮬레이션
2. 합성·구현·비트스트림 생성
3. FPGA 기록·동작 확인
4. 코드·사진·영상 업로드

사전 결과와 비교·고찰하고
GitHub 증빙 링크를 연결한다.

통합본 영상: 모드 전환·LCD 표시·선택한 회로의 동작.

레거시 01 · 논리 게이트

[실습 1] AND, OR, XOR 게이트 구현

- 입력 a, b에 대한 출력 x, y, z를 확인한다.
- 원본의 프로젝트 이름: logic_gate.
- 대상 부품: Spartan-7 xc7s75fgga484-1.

실습 순서

VS Code 사전 시뮬레이션·실험 전 레포트
Vivado 실습·FPGA 기록·동작 확인
사진·영상·GitHub·실험 후 레포트

작성일 2026. 09. 10. 이해리

원자료: 박동욱 교수, 서울시립대학교, 「논리 게이트 구현」.

다음 원문 페이지는 2022년 보관 PDF의 사진과 설명을 재구성했다.

원본: 논리 게이트 구현, p.3 레거시 01



서울시립대학교
UNIVERSITY OF SEOUL

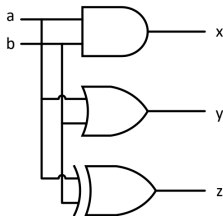
레거시 01 · 실습 목차

설계 기획	블록도·진리표
(1) 프로젝트 선언	부품·경로·프로젝트 설정
(2) 로직 설계	원본 코드·소스 등록
(3) Assignment	입출력 핀·전기 규격
(4) Compile	합성·배치배선
(5) Simulation	테스트벤치·파형
(6) Programming	비트스트림·실제 장치 기록
(7) 동작 확인	스위치 입력·LED 출력

원본 순서를 보존했다. 수업에서는 합성·구현에 앞서 VS Code와 Vivado에서 시뮬레이션한다. [원문](#) [보충 설명](#) · [실험 전·후 레포트](#) · [자료 안내](#)

설계 기획

블록도 및 진리표



서울시립대학교 전자전기컴퓨터공학부

입력		출력		
a	b	AND x	OR y	XOR z
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

원본: 논리 게이트 구현, p.4 레거시 01

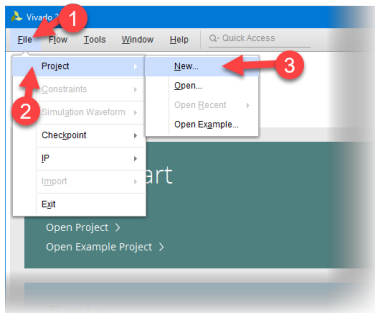


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(1) 프로젝트 선언

- Vivado 실행
- File > Project > New



원본: 논리 게이트 구현, p.5 레거시 01

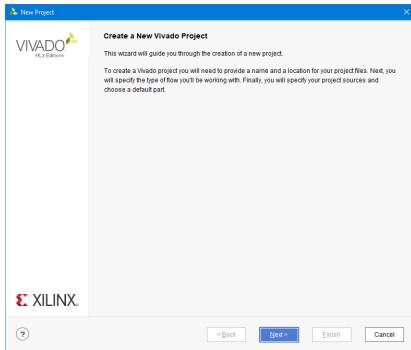


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(1) 프로젝트 선언

- Next



원본: 논리 게이트 구현, p.6 레거시 01

로직 설계 및 합성

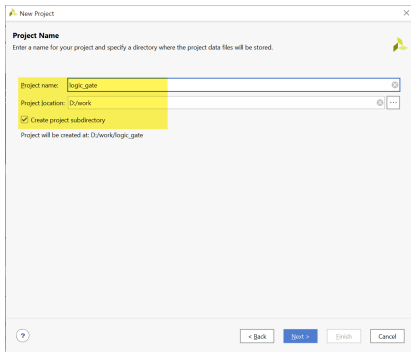
(1) 프로젝트 선언

- Project Name과 Project Location 설정
- Project Name : `logic_gate`
- Project location : `d:/work`
- Create project subdirectory : 체크
- `d:/work` 폴더에 `logic_gate` 폴더를 생성

로직 설계 및 합성

(1) 프로젝트 선언

원문 설명에 대응하는 실제 화면



New Project

Project Name
Enter a name for your project and specify a directory where the project data files will be stored.

Project name: logic_gate

Project location: D:/work

☒ Create project subdirectory

Project will be created at: D:/work/logic_gate

Back Next > Finish Cancel

원본: 논리 게이트 구현, p.7 레거시 01

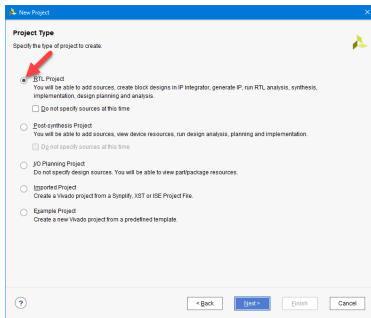


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(1) 프로젝트 선언

- Project Type 설정
- RTL Project



원본: 논리 게이트 구현, p.8 레거시 01

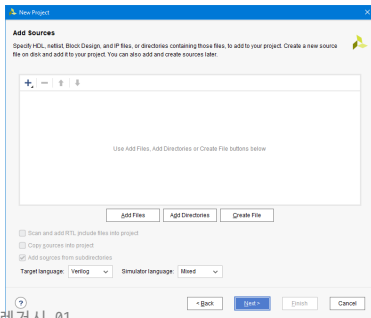


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(1) 프로젝트 선언

- Add source와 Add Constraints 설정
- 설계된 로직 파일과 설정된 핀 정보 파일이 없기 때문에 Next 버튼 누름.



원본: 논리 게이트 구현, p.9 레거시 01

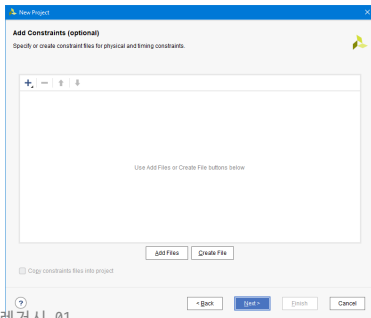


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(1) 프로젝트 선언

- Add source와 Add Constraints 설정
- 설계된 로직 파일과 설정된 핀 정보 파일이 없기 때문에 Next 버튼 누름.



원본: 논리 게이트 구현, p.9 레거시 01

로직 설계 및 합성

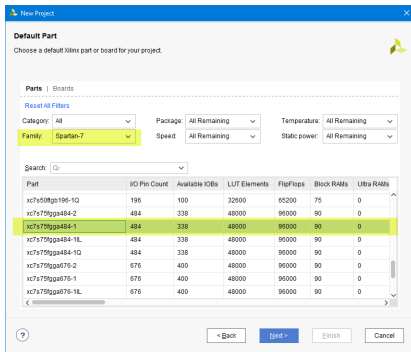
(1) 프로젝트 선언

- Device 설정
- Family : Spartan-7
- Part : xc7s75fgga484-1

로직 설계 및 합성

(1) 프로젝트 선언

원문 설명에 대응하는 실제 화면



원본: 논리 게이트 구현, p.10 레거시 01

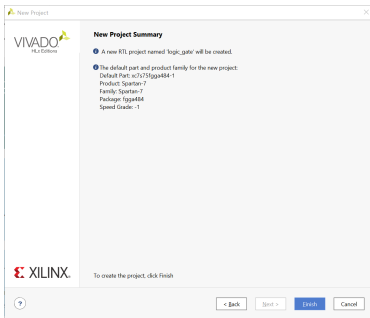


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(1) 프로젝트 선언

- 설정된 Summary 확인.
- Finish를 눌러 프로젝트 선언 완료



원본: 논리 게이트 구현, p.11 레거시 01

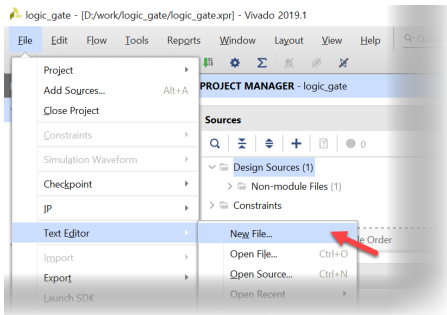


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(2) 로직 설계

- VIVADO 프로그램
- Text Editor > New File.. 선택



원본: 논리 게이트 구현, p.12 레거시 01

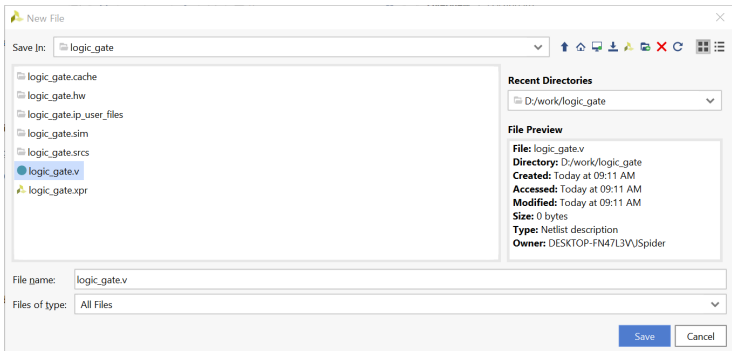


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(2) 로직 설계

- logic_gate.v 파일로 저장



원본: 논리 게이트 구현, p.13 레거시 01

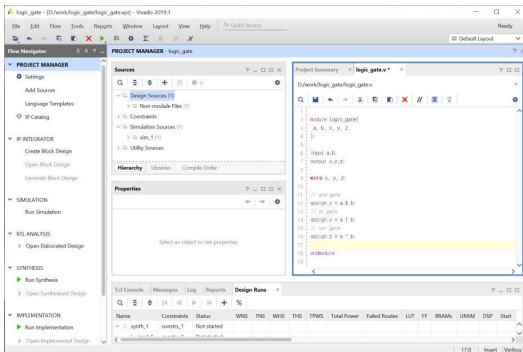


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(2) 로직 설계

- 로직 설계



원본: 논리 게이트 구현, p.14 레거시 01



서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(2) 로직 설계

- 로직 설계

```
1 |  
2 | module logic_gate(  
3 |     a, b, x, y, z  
4 | );  
5 |  
6 | input a,b;  
7 | output x,y,z;  
8 |  
9 | wire x, y, z;  
10 |  
11 | // and gate  
12 | assign x = a & b;  
13 | // or gate  
14 | assign y = a | b;  
15 | // xor gate  
16 | assign z = a ^ b;  
17 |  
18 | endmodule  
19 |
```

원본: 논리 게이트 구현, p.14 레거시 01

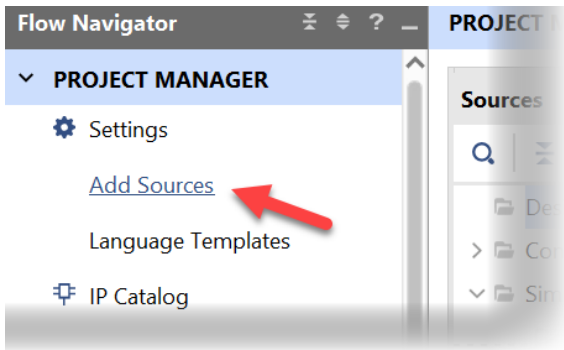


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(2) 로직 설계

- PROJECT MANAGER > Add Sources 선택



원본: 논리 게이트 구현, p.15 레거시 01

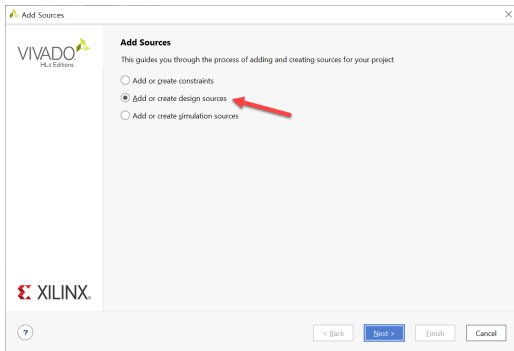


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(2) 로직 설계

- Add Sources 창에서 Add or Create design sources 선택

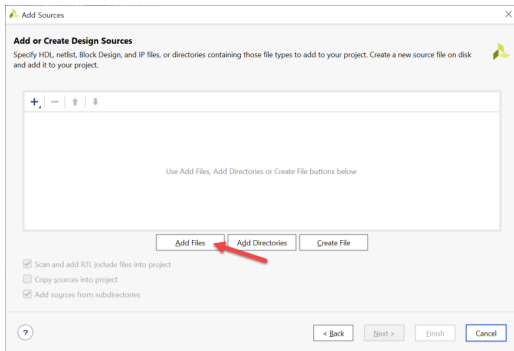


원본: 논리 게이트 구현, p.16 레거시 01

로직 설계 및 합성

(2) 로직 설계

- Add Files 버튼을 눌러, 앞의 logic_gate.v 파일 추가



원본: 논리 게이트 구현, p.17 레거시 01

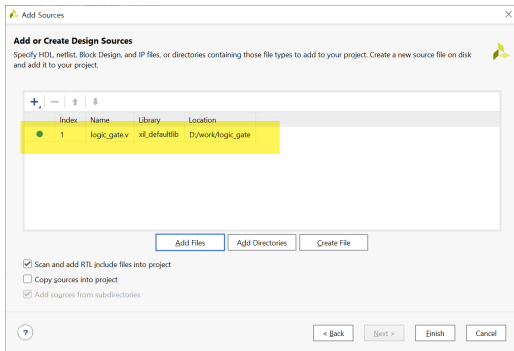


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(2) 로직 설계

- Add Files 버튼을 눌러, 앞의 logic_gate.v 파일 추가



원본: 논리 게이트 구현, p.17 레거시 01

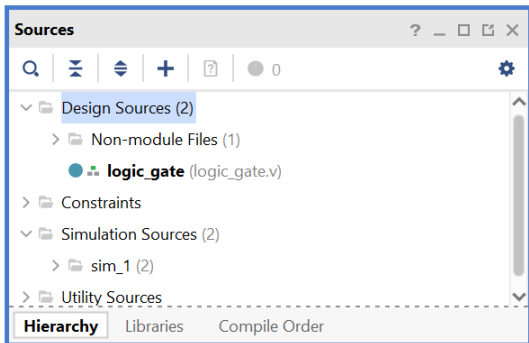


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(2) 로직 설계

- Sources 창에서 파일 추가된 것을 확인



원본: 논리 게이트 구현, p.18 레거시 01



서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

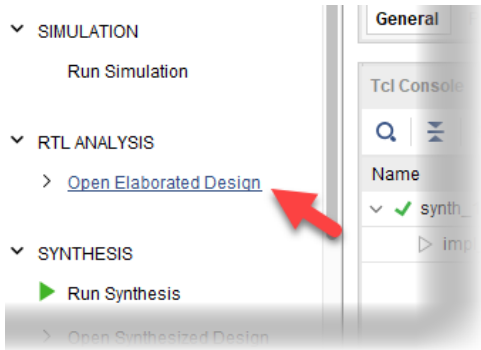
(3) Assignment

- 프로젝트의 타겟이 되는 FPGA 디바이스와 설계된 로직에서 사용하는 입출력 단자를 하드웨어와 연결하기 위하여 핀을 설정하는 것

로직 설계 및 합성

(3) Assignment

- PROJECT MANAGER의 RTL ANALYSIS 탭의 Open Elaborated Design 부분을 마우스로 클릭



원본: 논리 게이트 구현, p.20 레거시 01



서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

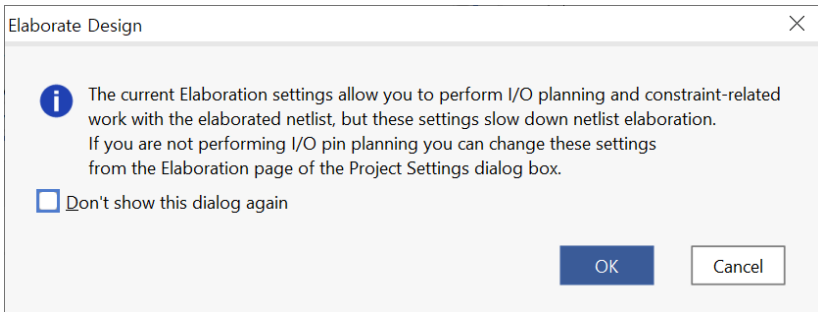
(3) Assignment

- 메시지 창이 나타난다.
- OK 버튼을 누르면 Elaboration 과정이 진행됨.
- 문법에 오류부분도 같이 체크하여 오류가 있다면 에러 메시지가 출력됨.

로직 설계 및 합성

(3) Assignment

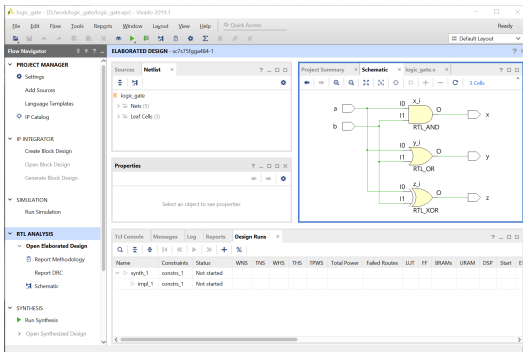
원문 설명에 대응하는 실제 화면



로직 설계 및 합성

(3) Assignment

- 설계된 Verilog HDL 구문이 Schematic으로 표현되는 것을 확인



원본: 논리 게이트 구현, p.22 레거시 01

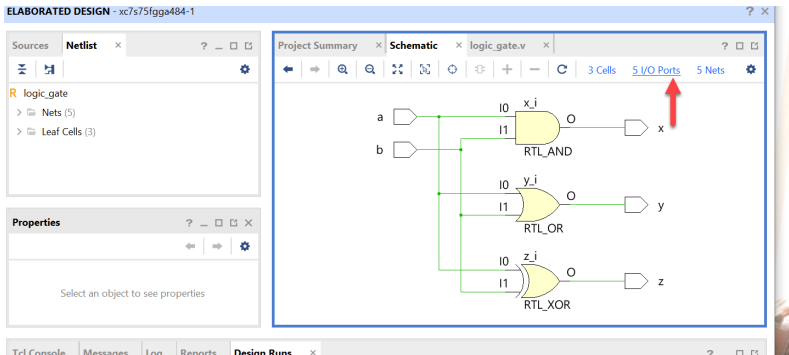


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(3) Assignment

- Schematic 화면 오른 쪽 윗 부분의 5 I/O Ports 선택



원본: 논리 게이트 구현, p.23 레거시 01



서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(3) Assignment

- I/O Std를 “LVCMOSS33” 으로 변경
- 입/출력 포트의 Voltage Level을 3.3V로 설정하는 것.

포트 이름 핀 번호			포트 이름 핀 번호		
a	K4	SW_1	x	L4	LED1
b	N8	SW_2	y	M4	LED2
			z	M2	LED3

원문 표기 유지. 실제 속성값은 **보충 설명**을 확인한다.

로직 설계 및 합성

(3) Assignment

- Package Pin 부분에 핀 설정
- I/O Std를 “LVCMOSS33” 으로 변경
- 입/출력 포트의 Voltage Level을 3.3V로 설정하는 것.

로직 설계 및 합성

(3) Assignment

원문 설명에 대응하는 실제 화면

Tcl Console Messages Log Reports Design Runs Find Results x											
Q [Icons]											
Name	Direction	Interface	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco	Vref	Drive Strength	
a	IN			K4	☒	35	LVC MOS33*	3.300			
b	IN			N8	☒	35	LVC MOS33*	3.300			
x	OUT			L4	☒	35	LVC MOS33*	3.300		12	
y	OUT			M4	☒	35	LVC MOS33*	3.300		12	
z	OUT			M2	☒	35	LVC MOS33*	3.300		12	

원본: 논리 게이트 구현, p.25 레거시 01

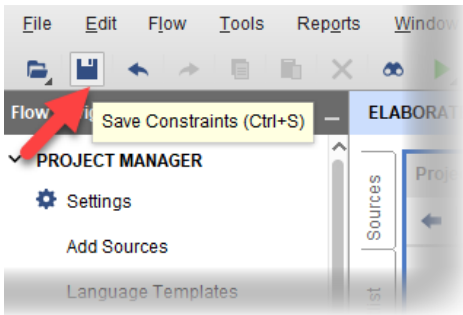


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(3) Assignment

- 핀 설정 저장
- 파일 명은 logic_gate로 설정



원본: 논리 게이트 구현, p.26 레거시 01

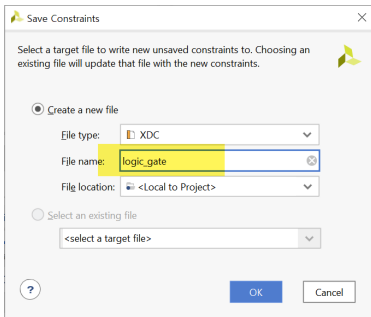


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(3) Assignment

- 핀 설정 저장
- 파일명은 logic_gate로 설정



원본: 논리 게이트 구현, p.26 레거시 01

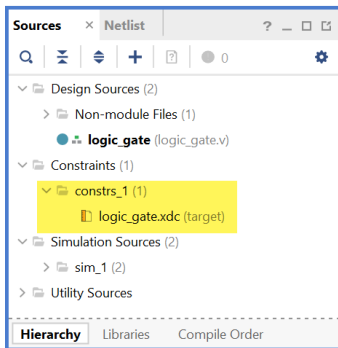


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(3) Assignment

- Sources 창에 추가된 것 확인



원본: 논리 게이트 구현, p.27 레거시 01

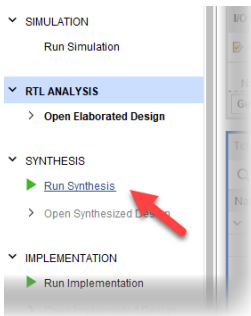


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(4) Compile

- SYNTHESIS > Run Synthesis
- 설계된 Verilog HDL 코드를 합성하여 최적화 시키는 과정



원본: 논리 게이트 구현, p.28 레거시 01

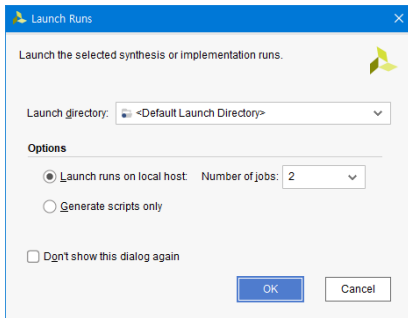


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(4) Compile

- SYNTHESIS > Run Synthesis
- 설계된 Verilog HDL 코드를 합성하여 최적화 시키는 과정



원본: 논리 게이트 구현, p.28 레거시 01

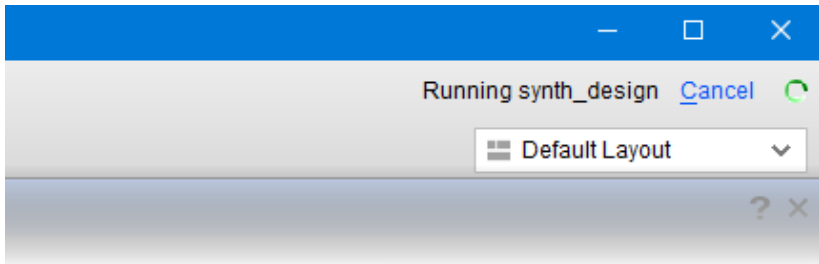


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(4) Compile

- Synthesis 동작 확인 위치
- 프로그램 상단 오른쪽 회전



로직 설계 및 합성

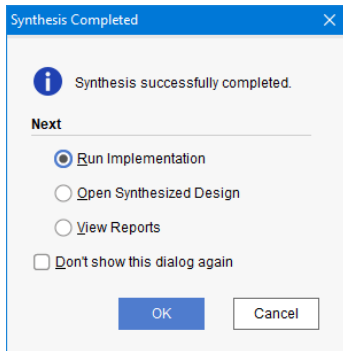
(4) Compile

- Synthesis 후 IMPLEMENTATION 진행
- Implementation
- 합성한 결과를 이용하여 실제 FPGA의 Logic Cell 단위로 바꾸어 FPGA내에 위치시키고 연결하는 Place & Route를 하는 과정

로직 설계 및 합성

(4) Compile

원문 설명에 대응하는 실제 화면



원본: 논리 게이트 구현, p.30 레거시 01

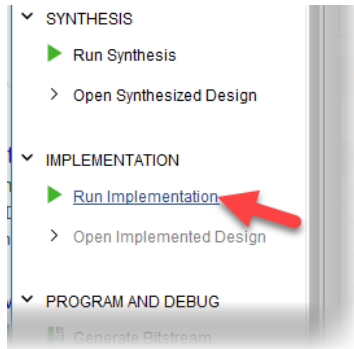


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(4) Compile

원문 설명에 대응하는 실제 화면



원본: 논리 게이트 구현, p.30 레거시 01

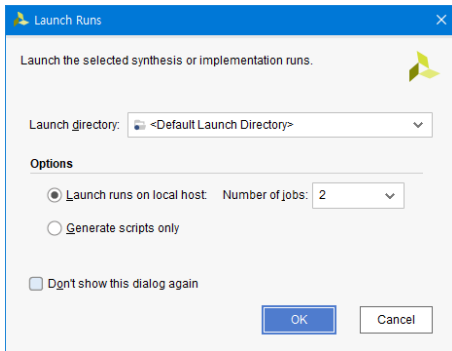


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(4) Compile

- Synthesis 후 IMPLEMENTATION 진행



원본: 논리 게이트 구현, p.31 레거시 01

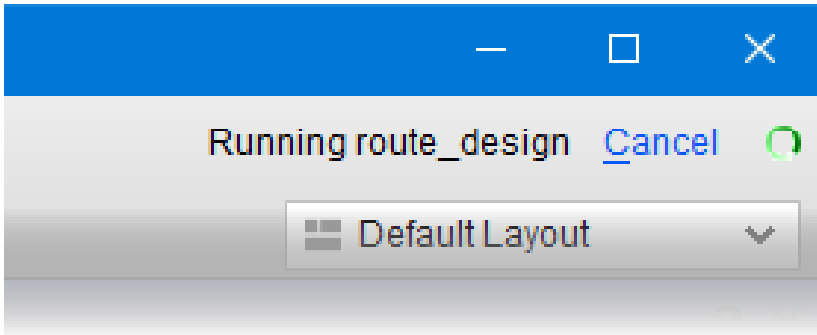


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(4) Compile

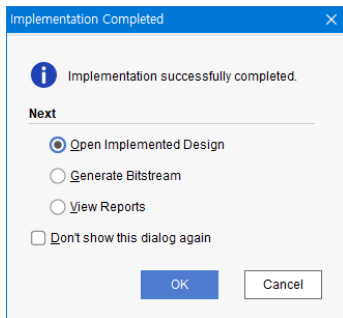
- Synthesis 후 IMPLEMENTATION 진행



로직 설계 및 합성

(4) Compile

- Implementation 이 끝난 화면
- Open Implemented Design 선택 후 OK



원본: 논리 게이트 구현, p.32 레거시 01

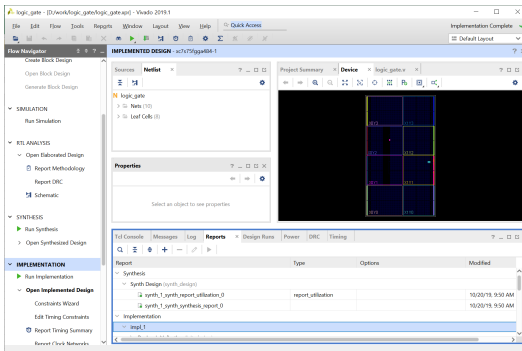


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(4) Compile

- Implementation 결과 확인



원본: 논리 게이트 구현, p.33 레거시 01



서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

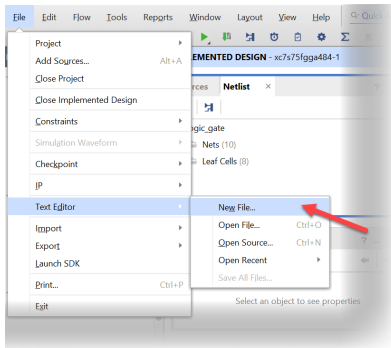
(5) Simulation

- 소프트웨어적으로 검증하는 과정
- 테스트 벤치 파일이 필요함.
- 시뮬레이션에 대한 입력 조건을 설정한 파일

로직 설계 및 합성

(5) Simulation

- File > Text Editor > New File 선택



원본: 논리 게이트 구현, p.35 레거시 01



서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(5) Simulation

- testbench.v 파일로 저장

```
1  `timescale 10ns / 100ps
2
3  module testbench();
4      // input
5      reg a, b;
6      // output
7      wire x, y, z;
8      // Instantiate the U1
9      logic_gate u1(a, b, x, y, z);
10     // Specify input stimulus
11     initial begin
12         a = 0; b = 0;
13
14         #20 a = 1;
15         #20 a = 0; b = 1;
16         #20 a = 1;
17     end
18
19 endmodule
20
```

원본: 논리 게이트 구현, p.36 레거시 01

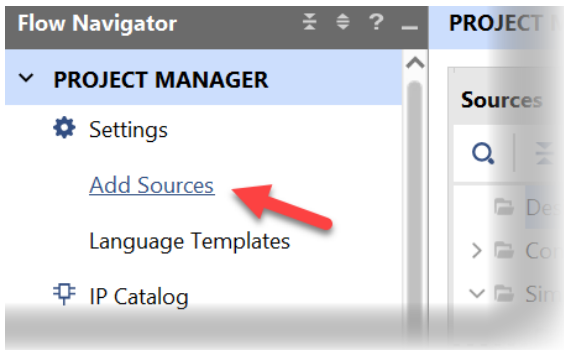


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(5) Simulation

- PROJECT MANAGER Add Source 선택



원본: 논리 게이트 구현, p.37 레거시 01

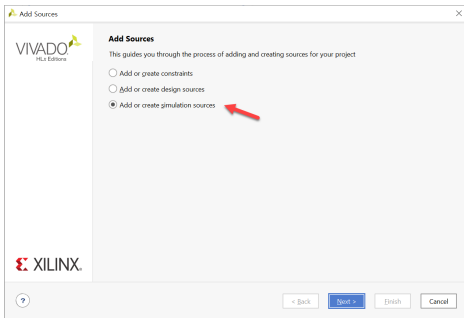


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(5) Simulation

- [Add or create simulation source] 선택
- Next

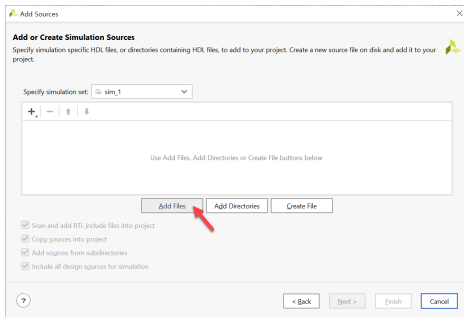


원본: 논리 게이트 구현, p.38 레거시 01

로직 설계 및 합성

(5) Simulation

- [Add Files]을 선택
- testbench.v 파일 선택



원본: 논리 게이트 구현, p.39 레거시 01

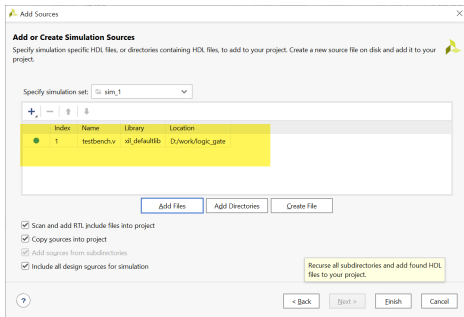


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(5) Simulation

- [Add Files]을 선택
- testbench.v 파일 선택



원본: 논리 게이트 구현, p.39 레거시 01

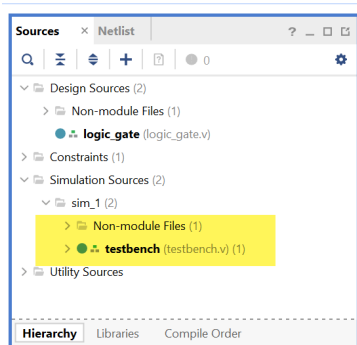


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(5) Simulation

- Simulation Sources 파일 추가 확인



원본: 논리 게이트 구현, p.40 레거시 01

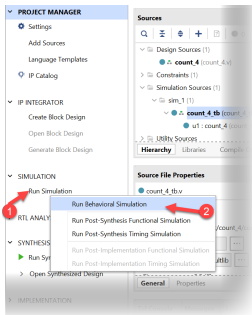


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(5) Simulation

- SIMULATION > Run Simulation
- Run Behavioral Simulation 실행



원본: 논리 게이트 구현, p.41 레거시 01

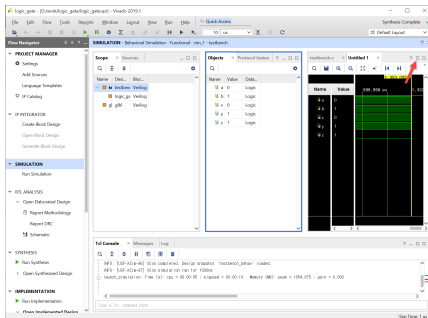


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(5) Simulation

- 결과 확인
- 상단 우측의 Maximize 버튼 클릭



원본: 논리 게이트 구현, p.42 레거시 01

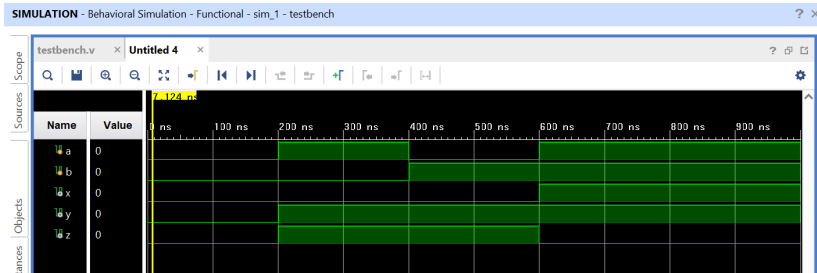


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(5) Simulation

- Zoom Fit 를 눌러 결과 확인



원본: 논리 게이트 구현, p.43 레거시 01

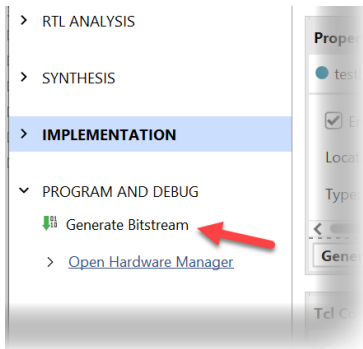


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(6) Programming

- Project Manager 창 PROGRAM AND DEBUG > Generate Bitstream



원본: 논리 게이트 구현, p.44 레거시 01

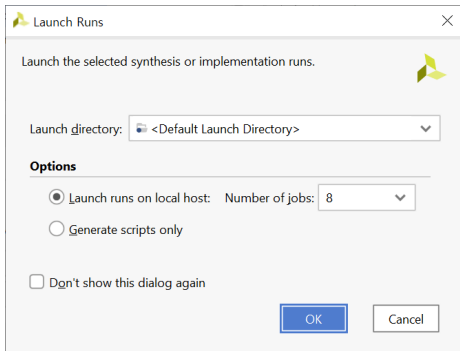


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(6) Programming

- Project Manager 창 PROGRAM AND DEBUG > Generate Bitstream



원본: 논리 게이트 구현, p.44 레거시 01

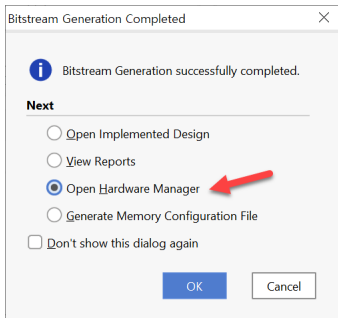


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(6) Programming

- 프로그래밍 파일 생성 완료 후
- Open Hardware Manager



원본: 논리 게이트 구현, p.45 레거시 01

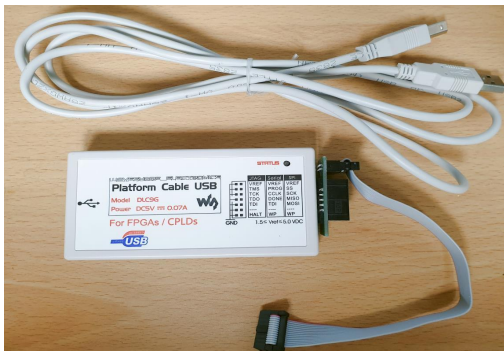


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(6) Programming

- 하드웨어 준비



원본: 논리 게이트 구현, p.46 레거시 01



서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

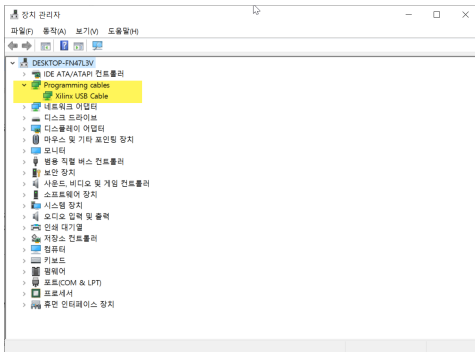
(6) Programming

- 장비의 전원이 꺼져 있는지 확인
- Xilinx Platform Cable에 USB Cable의 한 쪽 끝을 연결
- 반대쪽 끝은 PC의 USB 포트에 연결
- Xilinx Platform Cable에 연결된 2x14핀의 플랫 케이블은 장비의 Xilinx 모듈의 JTAG 포트에 연결
- 장비의 전원을 켜다.

로직 설계 및 합성

(6) Programming

- Xilinx Platform II Cable을 PC에 연결한 후, PC의 장치관리자에서 장치 인식 확인



원본: 논리 게이트 구현, p.48 레거시 01

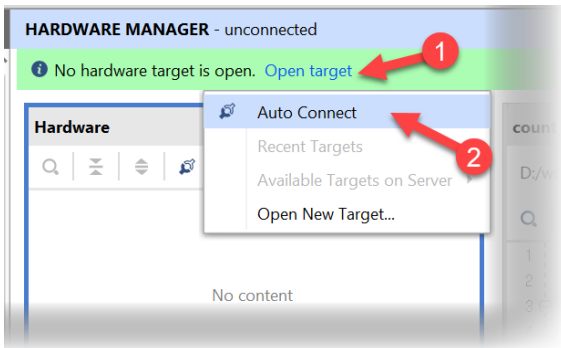


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(6) Programming

- Open Target > Auto Connect



원본: 논리 게이트 구현, p.49 레거시 01

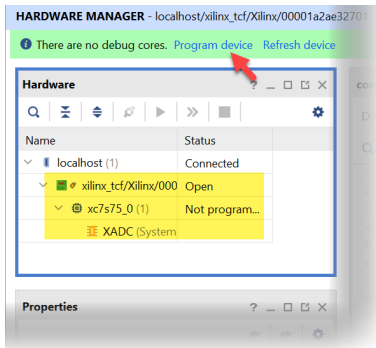


서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(6) Programming

- Program Device 선택



원본: 논리 게이트 구현, p.50 레거시 01



서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

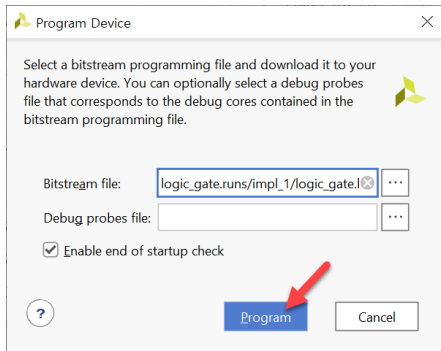
(6) Programming

- Bitstream File 선택
- <Project Folder> \ logic_gate.runs \ impl_1 \ logic_gate.bit 파일 선택
- Program 버튼 클릭

로직 설계 및 합성

(6) Programming

원문 설명에 대응하는 실제 화면



원본: 논리 게이트 구현, p.51 레거시 01



서울시립대학교
UNIVERSITY OF SEOUL

로직 설계 및 합성

(7) 동작 확인

- SW 1와 SW 2를 눌러 장비에서 어떻게 동작하는지 확인.

보충 · 원문 표기와 실행 조건

- 원본 p.24-25의 LVCM0SS33은 오타다.
실제 I/O 표준은 LVCM0S33을 선택한다.
- d:/work는 원본의 예시 작업 경로다.
실제 프로젝트를 저장한 경로를 사용한다.
- 원본 테스트벤치는 10 ns 단위에서 #20마다
입력을 바꾼다. 입력 전환 간격은 200 ns다.
- 원본의 a,b 입력 순서는 00, 10, 01, 11이다.
출력 x,y,z를 AND·OR·XOR 진리표와 비교한다.

이 페이지는 원본과 구분한 보충 설명이다. 원본 코드 이미지는 유지했다.

보충 · VS Code 사전 시뮬레이션

1. logic_gate.v와 테스트벤치를 VS Code에서 연다.
2. 선택한 시뮬레이터로 네 입력 조합을 모두 실행한다.
3. 원본 테스트벤치에는 자동 종료·VCD 저장 구문이 없다.
실행 시간을 지정하고 도구에 맞는 파형 저장 설정을 사용한다.
4. 생성한 파형에서 a,b,x,y,z를 확인하고 예상값과 비교한다.
5. 실행 방법·코드 설명·파형 해석을 실험 전 레포트에 쓴다.

VS Code 환경설정 매뉴얼 · [레포트 안내](#)

사전 단계는 이번 수업의 보충 안내이며 원본 Vivado 화면과 구분한다.

보충 · 보드 결과와 실험 후 레포트

1. Vivado 파형을 VS Code 사전 시뮬레이션과 비교한다.
2. 생성한 logic_gate.bit를 실제 FPGA에 기록한다.
3. SW_1·SW_2의 네 입력 조합과 LED1·LED2·LED3를 확인한다.
4. 연결 상태는 사진, 입력에 따른 출력 변화는 영상으로 남긴다.
5. 코드·파형·사진·영상을 GitHub에 올리고 레포트에 연결한다.

실제 결과와 예상값을 비교하고 오류·수정 사항을 설명한다.

원본 화면의 성공 표시와 본인의 실행 결과를 구분한다.

[레거시 01 처음으로](#) · [전체 목차 PDF](#)